

PROGRAMAÇÃO ORIENTADA A OBJETOS COM JAVA E WEB

Lista de Exercícios — Coleções e Programação Funcional

Prof. Dr. Antonio Marcos Selmini (selmini@fiap.com.br)

Exercício 01

Um navegador simplificado precisa oferecer os botões Voltar e Avançar. Cada vez que o usuário visita uma nova página, ela passa a ser a página atual; as páginas anteriores ficam disponíveis para o botão Voltar. Quando o usuário volta, a página que deixou de ser exibida fica disponível para o botão Avançar. Se o usuário visitar uma nova página depois de ter voltado, a lista de páginas para avançar é descartada.

Estrutura sugerida

```
public class Navegador {  
    private String paginaAtual;  
    private Deque<String> historicoVoltar;  
    private Deque<String> historicoAvancar;  
  
    public void visitar(String url) { ... }  
    public boolean voltar() { ... }  
    public boolean avancar() { ... }  
}
```

O que implementar

- Implemente `visitar(url)`: a página atual, se existir, vai para o topo de `historicoVoltar`; a nova URL passa a ser a atual; `historicoAvancar` é esvaziado.
- Implemente `voltar()`: se houver páginas para voltar, a atual vai para `historicoAvancar` e o topo de `historicoVoltar` passa a ser a atual. Devolva `false` quando não houver para onde voltar.
- Implemente `avancar()` seguindo a mesma ideia do `voltar()`.
- Na classe principal, simule a sequência: visitar A, B, C; voltar; voltar; avancar; visitar D; voltar; avancar; avancar. Após cada operação, exiba a página atual e a quantidade de páginas em cada pilha.
- Exiba, ao final, o conteúdo completo de `historicoVoltar` sem removê-lo, utilizando `forEach` com method reference.

Restrições e pontos de atenção

- Utilize `Deque<String>` declarado pela interface, com `ArrayDeque` como implementação. Não use `Stack`. Não falei sobre `Stack` e `Deque` em aula, mas vocês viram com a Patrícia. Pela documentação da linguagem vocês conseguem utilizar
- Utilize apenas os métodos de pilha (`push`, `pop`, `peek`, `isEmpty`, `clear`). Não acesse os elementos por índice.
- Após a operação visitar D, a chamada `avancar()` deve devolver `false` — verifique se o seu esvaziamento está correto.

Exercício 02

Um aplicativo de música guarda a playlist do usuário como uma lista de faixas. Cada faixa possui título, artista, duração em segundos, ano de lançamento e a quantidade de vezes que foi reproduzida.

Estrutura sugerida

```
public class Faixa {  
    private String titulo;  
    private String artista;  
    private int duracaoSegundos;  
    private int ano;  
    private int reproducoes;  
}
```

O que implementar

- Crie uma `List<Faixa>` de ao menos quatro artistas diferentes e com alguns anos repetidos.
- Exiba a playlist ordenada por artista e, dentro do mesmo artista, por ano decrescente. Utilize `Comparator.comparing` com `thenComparing`.
- Exiba as cinco faixas mais reproduzidas, da mais reproduzida para a menos reproduzida, mostrando somente título e quantidade de reproduções.
- Remova da playlist todas as faixas com menos de 3 reproduções e exiba quantas foram removidas. Dica: pode ser utilizado o `removeIf`.
- Calcule e exiba a duração total da playlist restante no formato `hh:mm:ss`. A soma deve ser feita com `mapToInt` e `sum`; a formatação, na classe principal.

Restrições e pontos de atenção

- O item 2 exige `Comparator.comparing(Faixa::getArtista).thenComparing(Faixa::getAno, Comparator.reverseOrder())` ou equivalente. Observe que `.reversed()` aplicado ao final inverteria também a ordem dos artistas.
- A lista original não pode ser alterada pelas ordenações dos itens 2 e 3 — use `sorted()` em um Stream ou trabalhe sobre uma cópia.

Exercício 03

A organização de um congresso recebe inscrições por vários canais e o mesmo participante acaba cadastrado mais de uma vez, às vezes com o e-mail digitado com letras maiúsculas ou espaços extras. Dois convidados são considerados a mesma pessoa quando possuem o mesmo e-mail, desconsiderando maiúsculas, minúsculas e espaços nas extremidades.

Estrutura sugerida

```
public class Convidado {  
    private String nome;  
    private String email;  
    private String empresa;  
}
```

O que implementar

- Sobrescreva `equals()` e `hashCode()` em `Convidado` considerando apenas o e-mail normalizado para minúsculo.
- Crie uma `List<Convidado>` contendo ao menos quatro duplicatas disfarçadas ("Ana@Empresa.com", "ana@empresa.com" etc.).
- Converta a lista em um `Set<Convidado>` e exiba a quantidade de registros antes e depois da remoção de duplicatas.
- Dada uma segunda coleção, `Set<Convidado> confirmados`, exiba com `retainAll` quais inscritos confirmaram presença e, com `removeAll`, quais ainda não confirmaram.

Restrições e pontos de atenção

- Não altere a lista original para remover duplicatas; a remoção deve ser consequência do contrato de `Set`.
- O `hashCode()` deve ser coerente com o `equals()`: teste incluindo o mesmo e-mail com grafias diferentes.

Exercício 04

O formulário de cadastro de um aplicativo precisa validar os dados informados pelo usuário. Em vez de uma sequência de `if`, as regras devem ser modeladas como interfaces funcionais reutilizáveis e combináveis.

Estrutura sugerida

```
public class Cadastro {
    private String nome;
    private String email;
    private String senha;
    private int idade;
}

@FunctionalInterface
public interface Regra {
    String validar(Cadastro c);    // devolve null quando válido ou a mensagem de erro
}
```

O que implementar

- Declare, como variáveis do tipo `Predicate<Cadastro>`, as regras: nome com ao menos 3 caracteres; e-mail contendo `@` e `.`; senha com ao menos 8 caracteres; senha contendo ao menos um dígito; idade entre 18 e 120.
- Combine as regras com `and()`, `or()` e `negate()` para construir: `cadastroValido` (todas as regras) e `senhaFraca` (senha curta OU sem dígito). Exiba o resultado dos dois predicados para pelo menos cinco cadastros de teste.
- Implemente a interface `Regra` com lambdas, monte uma `List<Regra>` e escreva um método `List<String> erros(Cadastro c)` que percorra a lista e devolva todas as mensagens de erro (não apenas a primeira).

Restrições e pontos de atenção

- Neste exercício não se usa Stream: o objetivo é dominar as interfaces de `java.util.function` e a composição de funções.
- Nenhum `if` pode aparecer nas regras de validação — as condições devem estar dentro das lambdas.
- O item 4 exige diferenciar `Predicate` (resposta sim/não) de `Regra` (resposta com mensagem). Comente por que uma interface própria foi necessária.

Exercício 05

A secretaria acadêmica controla as matrículas de alunos em disciplinas. Um aluno pode estar matriculado em várias disciplinas e cada matrícula registra a nota final obtida. Um aluno é aprovado em uma disciplina quando a nota é maior ou igual a 6,0. Uma disciplina só é aberta quando tem ao menos três matriculados.

Estrutura sugerida

```
public class Aluno {           // igualdade pelo RM
    private String rm;
    private String nome;
    private String curso;
}

public class Disciplina {     // igualdade pelo código
    private String codigo;
    private String nome;
    private int semestreSugerido;
}

public class Matricula {
    private Aluno aluno;
    private Disciplina disciplina;
    private double nota;
}
```

O que implementar

- Sobrescreva `equals()` e `hashCode()` em `Aluno` (pelo RA) e em `Disciplina` (pelo código).
- Crie alguns alunos de dois cursos, de algumas disciplinas e algumas matrículas. Garanta que uma disciplina tenha menos de três matriculados e que ao menos um aluno tenha nota abaixo de 6 em duas disciplinas.
- Monte um `Map<Disciplina, Set<Aluno>>` com os matriculados de cada disciplina. Exiba somente as disciplinas abertas, com os alunos em ordem natural (escolha a implementação de `Set` que garanta isso).
- Monte um `Map<String, Double>` contendo a média das notas por curso, usando `groupingBy` com `averagingDouble`. Exiba as médias com duas casas decimais.
- Monte um `Map<Aluno, List<String>>` com os nomes das disciplinas em que cada aluno foi reprovado, usando `groupingBy` com `Collectors.mapping`. Exiba somente os alunos com duas ou mais reprovações.

Restrições e pontos de atenção

- `averagingDouble` devolve 0.0 para um grupo vazio, o que pode ser enganoso; garanta no item 4 que só cursos com matrícula apareçam no mapa.
- A chave `Aluno` do item 6 só funciona porque `equals()/hashCode()` foram sobrescritos. Teste o que acontece se você os remover e registre a observação em comentário.

Saída esperada (fragmento ilustrativo)

- Disciplina POO2 — 4 matriculados: Ana Lima, Bruno Costa, Carla Souza, Diego Ramos
- Média por curso: SI = 7,15 | ADS = 6,80
- Aprovados: 15 (75,0%) | Reprovados: 5 (25,0%)
- Alunos com 2+ reprovações: Diego Ramos → [Estruturas de Dados, Banco de Dados]
- Melhor aluno: Ana Lima (SI) — média 9,10

Exercício 06

Uma plataforma de vídeos registra cada visualização feita pelo usuário. Cada vídeo tem título, canal, duração em minutos e um conjunto de tags (por exemplo, "`java`", "`backend`", "`carreira`"). Cada visualização registra o vídeo assistido e o percentual assistido (0 a 100). A plataforma mantém uma lista de "assistidos recentemente" com no máximo cinco vídeos, sem repetição, em que o mais recente aparece primeiro.

Estrutura sugerida

```
public class Video {           // igualdade pelo título + canal
    private String titulo;
    private String canal;
    private int duracaoMinutos;
    private Set<String> tags;
}

public class Visualizacao {
    private Video video;
    private int percentualAssistido;
}

public class Historico {
    private List<Visualizacao> visualizacoes = new ArrayList<>();
    private Deque<Video> recentes = new ArrayDeque<>();
    public void registrar(Video v, int percentual) { ... }
}
```

O que implementar

- Implemente `registrar`: adiciona a visualização à lista e atualiza `recentes` — se o vídeo já estiver no `Deque`, ele é removido da posição antiga e inserido no início; se o `Deque` ultrapassar cinco elementos, o mais antigo é descartado.
- Crie alguns vídeos de quatro canais, com tags que se repitam entre vídeos, e registre visualizações, incluindo o mesmo vídeo assistido mais de uma vez.
- Exiba os vídeos assistidos recentemente, do mais recente para o mais antigo, com `forEach` e `method reference`.

